*Università degli Studi di Napoli – Federico II*

# *Progetto di contatori*

***Richiami e cenni di implementazione in VHDL***

**ARCHITETTURA** *dei* **SISTEMI DIGITALI**

*Prof.ssa Alessandra De Benedictis*

# *Contatori*

- Nella sua forma più semplice, un contatore è una macchina di Moore priva di ingressi dato, che evolve attraverso una sequenza predefinita di **stati di conteggio** in corrispondenza di un **segnale di conteggio**
  - Il segnale di conteggio deve avere una durata abbastanza breve da evitare transizioni multiple di stato;
  - in alternativa, la macchina deve essere sensibile ai fronti del segnale di conteggio.

- Un **contatore mod-n** possiede n stati, a ciascuno dei quali è associato un valore di conteggio fra 0 ed n-1.
  - Oltre al segnale di conteggio e all'uscita di conteggio, un contatore potrebbe essere dotato di segnali di ingresso aggiuntivi quali segnale di *abilitazione*, *reset*, e *preset*, e di una uscita cosiddetta *divisore*, attiva quando si raggiunge il valore massimo di conteggio

# *Approcci al progetto di contatori*

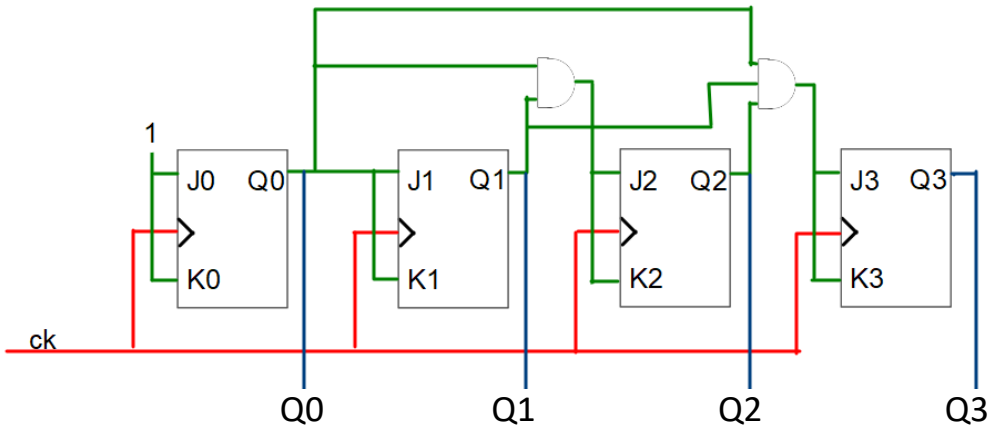
- Un contatore può essere progettato seguendo due approcci principali:
  - **Progetto come macchina sequenziale sincrona** (approccio metodologico): si modella il comportamento della macchina con un automa (Moore) e si procede alla sintesi mediante posizionamento dei flip-flop.
    - L'automa avrà un numero di stati pari al modulo del contatore, che potrebbe facilmente diventare ingestibile.
  - **Progetto per composizione/riduzione di componenti**: si individuano dei componenti che realizzano un sottoinsieme/sovrainsieme delle funzionalità desiderate e si identificano i collegamenti fra tali componenti e/o gli ingressi da applicare.
    - Per realizzare un contatore *classico* tipicamente si utilizzano contatori di modulo inferiore opportunamente interconnessi oppure un contatore di modulo superiore a cui viene applicato un segnale di reset prima della fine del conteggio.
    - Per realizzare un contatore più complesso, ad esempio con incrementi di valore variabile, è possibile utilizzare componenti diversi, come un addizionatore

# *Progetto come macchina sequenziale sincrona (1/2)*

| Present state | | | | Next state | | | |
|---|---|---|---|---|---|---|---|
| $Q_3$ | $Q_2$ | $Q_1$ | $Q_0$ | $Q_3$ | $Q_2$ | $Q_1$ | $Q_0$ |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |
| 0 | 0 | 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 1 | 1 | 0 | 1 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 | 1 |
| 0 | 1 | 0 | 1 | 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | 0 | 0 | 1 |
| 1 | 0 | 0 | 1 | 1 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 | 0 | 1 | 1 |
| 1 | 0 | 1 | 1 | 1 | 1 | 0 | 0 |
| 1 | 1 | 0 | 0 | 1 | 1 | 0 | 1 |
| 1 | 1 | 0 | 1 | 1 | 1 | 1 | 0 |
| 1 | 1 | 1 | 0 | 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |

| Present state | | | | Next state | | | | Flip-flop inputs | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $Q_3$ | $Q_2$ | $Q_1$ | $Q_0$ | $Q_3$ | $Q_2$ | $Q_1$ | $Q_0$ | $J_{Q3}$ | $K_{Q3}$ | $J_{Q2}$ | $K_{Q2}$ | $J_{Q1}$ | $K_{Q1}$ | $J_{Q0}$ | $K_{Q0}$ |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | × | 0 | × | 0 | × | 1 | × |
| 0 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | × | 0 | × | 1 | × | × | 1 |
| 0 | 0 | 1 | 0 | 0 | 0 | 1 | 1 | 0 | × | 0 | × | × | 0 | 1 | × |
| 0 | 0 | 1 | 1 | 0 | 1 | 0 | 0 | 0 | × | 1 | × | × | 1 | × | 1 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | × | × | 0 | 0 | × | 1 | × |
| 0 | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | × | × | 0 | 1 | × | × | 1 |
| 0 | 1 | 1 | 0 | 0 | 1 | 1 | 1 | 0 | × | × | 0 | × | 0 | 1 | × |
| 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 1 | × | × | 1 | × | 1 | × | 1 |
| 1 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | × | 0 | 0 | × | 0 | × | 1 | × |
| 1 | 0 | 0 | 1 | 1 | 0 | 1 | 0 | × | 0 | 0 | × | 1 | × | × | 1 |
| 1 | 0 | 1 | 0 | 1 | 0 | 1 | 1 | × | 0 | 0 | × | × | 0 | 1 | × |
| 1 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | × | 0 | 1 | × | × | 1 | × | 1 |
| 1 | 1 | 0 | 0 | 1 | 1 | 0 | 1 | × | 0 | × | 0 | 0 | × | 1 | × |
| 1 | 1 | 0 | 1 | 1 | 1 | 1 | 0 | × | 0 | × | 0 | 1 | × | × | 1 |
| 1 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | × | 0 | × | 0 | × | 0 | 1 | × |
| 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | × | 1 | × | 1 | × | 1 | × | 1 |

# *Progetto come macchina sequenziale sincrona (1/2)*




$J_0 = K_0 = 1$

$J_1 = K_1 = Q_0$

$J_2 = K_2 = Q_0\ Q_1$

$J_3 = K_3 = Q_0\ Q_1\ Q_2$

- Il flip-flop cui corrisponde il bit meno significativo commuta ad ogni fronte attivo del clock

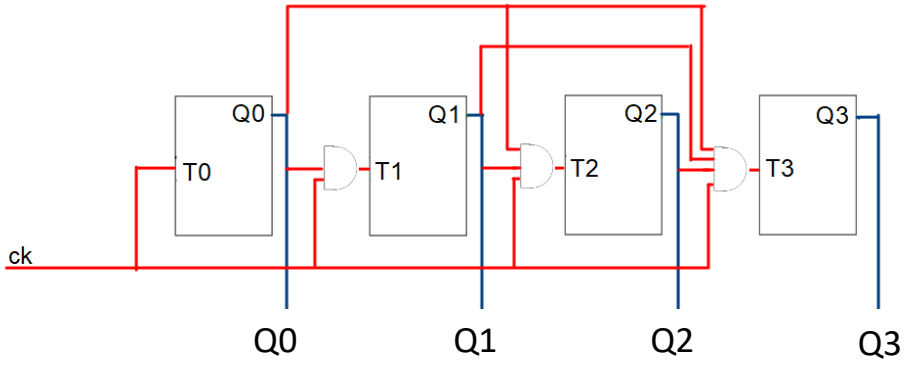
- Ciascuno dei flip-flop successivi commuta in corrispondenza del fronte di attivo del clock, solo quando tutti i bit delle posizioni precedenti meno significative sono uguali a 1

## *Progetto per composizione (1/2)*

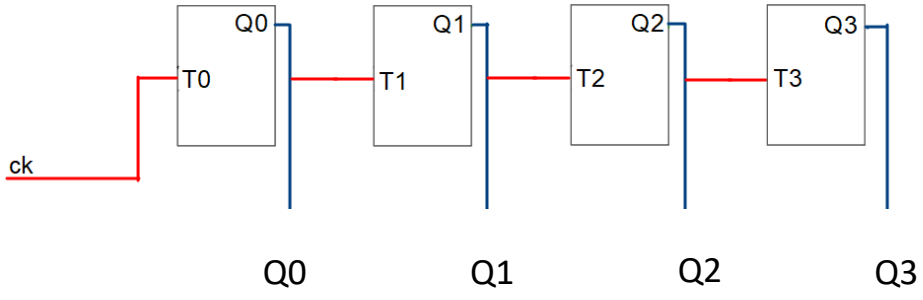
- Quando si compongono diversi contatori per realizzarne uno di modulo complessivamente più grande, è possibile adottare due diverse strategie di collegamento:
  - **schema parallelo**: Il segnale di conteggio è fornito simultaneamente a tutti i contatori (flip-flop) interconnessi, e la diversa frequenza di conteggio è assicurata da una opportuna combinazione del segnale di conteggio e delle uscite dei contatori (flip-flop) precedenti.
  - **schema seriale** (o a cascata): il segnale di conteggio è fornito solo al contatore di peso meno significativo, mentre per tutti gli altri è fornito dalle uscite dei contatori precedenti.
    - In particolare, ogni contatore (dal secondo in poi) riceve un impulso di conteggio quando quello precedente raggiunge il suo valore massimo di conteggio
    - Il comportamento della macchina complessivamente è asincrono

# *Progetto per composizione (2/2)*

- Nella configurazione **parallela**, il clock viene dato a tutti i flip-flop a viene opportunamente manipolato per garantire la diversa frequenza di commutazione;
- Nella configurazione **serie** il segnale di conteggio viene dato solo al primo flip-flop, e ogni flip-flop a partire dal secondo prende l'impulso di conteggio dall'uscita del precedente
  - Lo schema seriale implica una **propagazione** degli impulsi di conteggio lungo la catena di contatori, e quindi per effetto dei ritardi ci saranno degli istanti in cui il valore complessivo di conteggio non è corretto



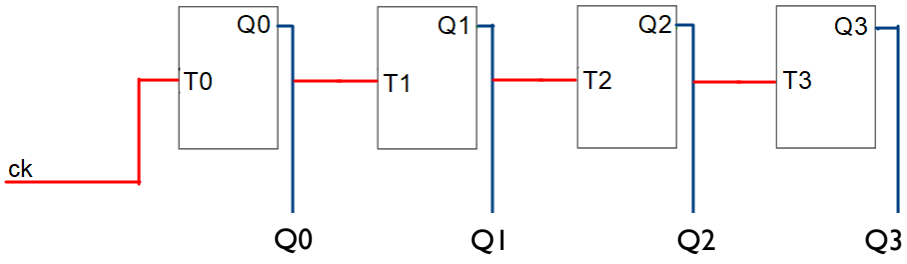

SCHEMA PARALLELO

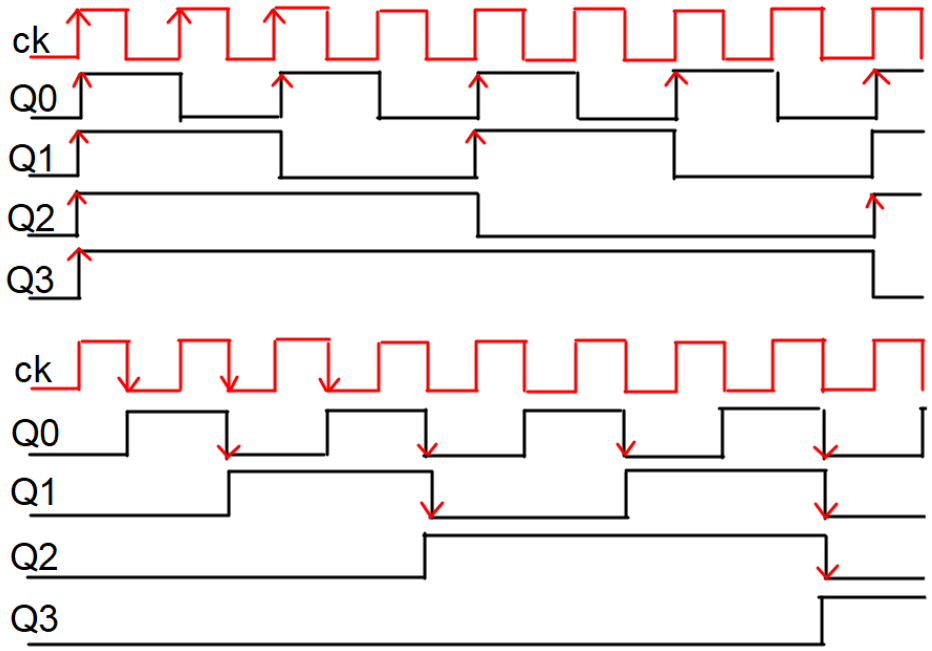



SCHEMA SERIALE

# *Tempificazione del segnale di conteggio*

- Nelle configurazioni parallela e seriale è necessario utilizzare flip-flop di tipo **edge-triggered sul fronte di discesa**
  - Devono essere edge-triggered perché devono commutare solo in istanti ben precisi, e devono essere sul fronte di discesa perché altrimenti appena commuta il primo commuterebbe anche il secondo e così via (e avremmo un conteggio a decrescere)




Nota: in realtà il flip-flop meno significativo potrebbe commutare anche sul fronte di salita (mentre tutti gli altri devono commutare solo sul fronte di discesa), ma si preferisce una tempificazione uniforme!

# Implementazione VHDL di un contatore: es. contatore mod-16 schema seriale

```
1   library IEEE;
2   use IEEE.STD_LOGIC_1164.ALL;
3
4   entity ffT is
5       port(
6           clk: in std_logic;
7           reset: in std_logic;
8           Y: out std_logic
9           );
10  end  ffT;
11
12  architecture rtl of ffT is
13
14      signal TY: std logic;
15
16      begin
17      ff: process(clk,reset)
18          begin
19              if(reset='1') then
20                  TY<='0';
21              elsif(clk'event AND clk='0') then
22                  TY<=not TY;
23              end if;
24          end process;
25
26      Y<=TY;
27
28  end rtl;
```

Implementiamo il flip-flop T utilizzando una descrizione comportamentale

```
5   entity cont_16_s is
6       port(
7           C: in std_logic;
8           R: in std_logic;
9           cont: out std_logic_vector(0 to 3)
10          );
11      end cont_16_s;
12
13  architecture structural of cont_16_s is
14
15      component ffT is
16          port(
17              clk: in std_logic;
18              reset: in std_logic;
19              Y: out std_logic
20              );
21      end  component;
22
23      signal S1: std_logic;
24      signal S2: std_logic;
25      signal S3: std_logic;
26      signal S4: std_logic;
27
28      begin
29      ff1: ffT
30          port map(
31                  C,
32                  R,
33                  S1
34              );
35      ff2: ffT
36          port map(
37                  S1,
38                  R,
39                  S2
40              );
41      ff3: ffT
42          port map(
43                  S2,
44                  R,
45                  S3
46              );
47      ff4: ffT
48          port map(
49                  S3,
50                  R,
51                  S4
52              );
53
54      cont<=S4 & S3 & S2 & S1;
55  end structural;
```

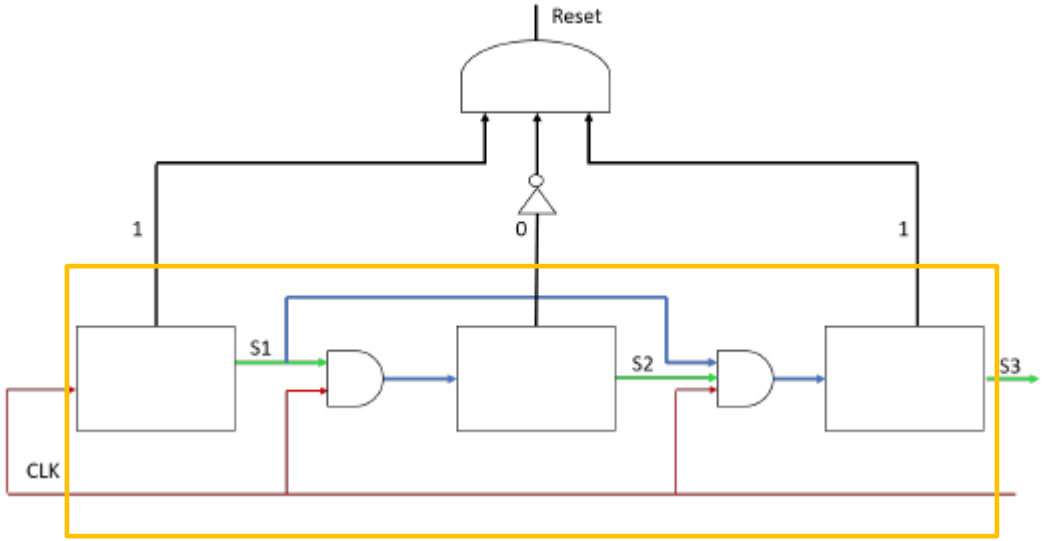
L'operatore '&' consente di effettuare una concatenazione dei segnali che compongono l'uscita di conteggio

I segnali S1-S4 sono utilizzati per effettuare le connessioni fra i flip-flop componenti

## *Progetto di contatori per riduzione*

- Il progetto di un contatore mod-N per **riduzione** (anche detto in modulo diminuito) può essere effettuato utilizzando un contatore mod-M con M=2n>N e **sfruttando il segnale di reset** o, laddove presente, il segnale di **load**, che consente di precaricare un valore specifico.
- Sia il segnale di reset che quello di load possono essere sincroni o asincroni:
  - il segnale è detto *sincrono* quando il suo valore ha effetto sullo stato di un sistema solo in corrispondenza del fronte attivo del segnale di conteggio
  - il segnale è detto *asincrono* quando la variazione del suo valore causa una variazione dello stato del sistema indipendentemente dal segnale di conteggio

# *Progetto di contatori per riduzione: es. contatore mod-6 da mod-8*

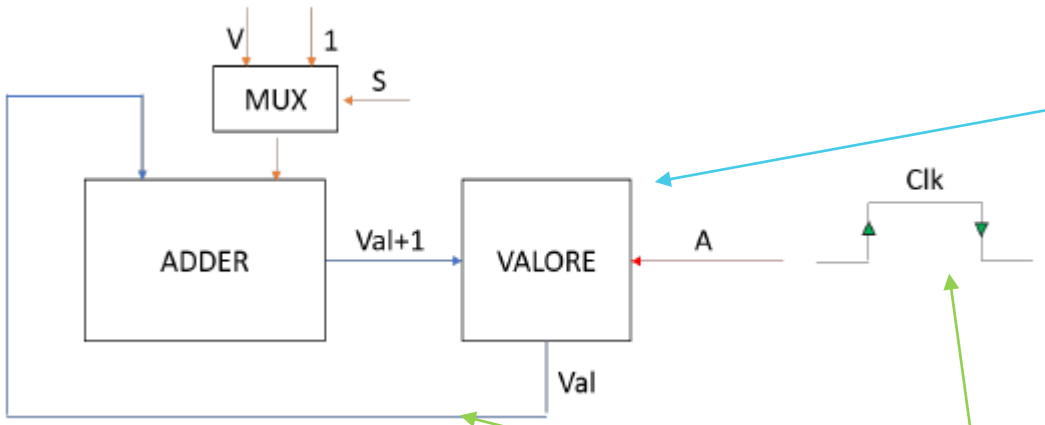



- Per realizzare un contatore mod-6 da uno mod-8 utilizzando un *reset sincrono*, è necessario alzare il segnale di reset quando il conteggio raggiunge il valore 5 (101).
- Al successivo clock, il reset si abbasserà e il contatore effettivamente si resetterà, ricominciando il suo conteggio dal valore 0.
  - Attenzione: ciò implica che il segnale di **reset sia attivo sul fronte di discesa**!

# *Progetto di un contatore ad incremento variabile*

Un contatore ad incremento variabile può essere facilmente implementato utilizzando un adder che, in ogni stato, incrementa il valore di uscita *Val* del valore dell'incremento *V.*

Un MUX può essere utilizzato per selezionare l'incremento V o il classico incremento unitario




Un elemento di memoria memorizza il valore corrente della somma

Ad ogni fronte di salita del clock, la somma parziale viene retroazionata in ingresso al sommatore. Il fronte di discesa può essere utilizzato per leggere il valore corrente della somma, assicurandosi che l'intervallo di tempo fra i due fronti sia sufficiente a completare la somma corrente

# *Implementazione VHDL di tipo comportamentale di un contatore mod-4*

Reset asincrono

```
entity counters_1 is
    port(C, CLR : in std_logic;
    Q : out std_logic_vector(3 downto 0));
end counters_1;
architecture archi of counters_1 is
    signal tmp: std_logic_vector(3 downto 0);
    begin
        process (C, CLR)
            begin
            if (CLR='1') then
            tmp <= "0000";
            elsif (C'event and C='1') then
            tmp <= tmp + 1;
            end if;
        end process;
        Q <= tmp;
end archi;
```

Reset sincrono

```
architecture archi of counters_1 is
    signal tmp: std_logic_vector(3 downto 0);
    begin
        process (C, CLR)
            begin
            if (C'event and C='1') then
                if (CLR='1') then
                    tmp <= "0000";
                else
                    tmp <= tmp + 1;
                end if;
            end if;
        end process;
        Q <= tmp;
end archi;
```